

HDOC DAY - 31

NAME: Yug Nirwal

SAP ID: 590033354

SUBJECT: C programming

DATE: 22-09-26

2D Arrays - Menu Driven Matrix Operations

C Programming | Hands-on Coding Activity

Problem Statement

Write a C program to perform the following menu-driven operations on a given 2D array:

1. Find the sum of all elements in a matrix.
2. Find the sum of each row of a matrix and store it in an array.
3. Find the transpose of a matrix.
4. Exit.

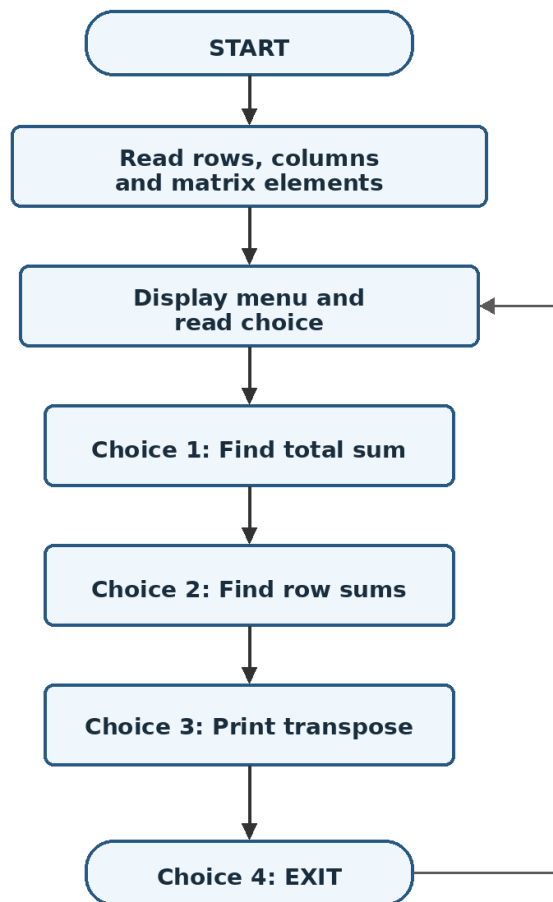
Aim

To understand how a two-dimensional array can be processed using nested loops and a menu-driven C program. The program calculates the total of all matrix elements, stores row-wise sums in a separate array, and displays the transpose of the matrix.

Algorithm

5. Start the program and read the number of rows and columns.
6. Read all matrix elements using nested loops.
7. Display a menu with four choices.
8. For choice 1, traverse the complete matrix and add every element to a total sum.
9. For choice 2, calculate each row sum separately and store it in the row-sum array.
10. For choice 3, interchange the row and column positions while displaying the matrix to obtain the transpose.
11. For an invalid choice, display an appropriate message.
12. Repeat the menu until choice 4 is selected, then stop.

Flowchart



C Program

```
#include <stdio.h>

int main() {
    int a[10][10], rowSum[10];
    int rows, cols, choice;

    printf("Enter the number of rows and columns: ");
    scanf("%d %d", &rows, &cols);

    if (rows <= 0 || rows > 10 || cols <= 0 || cols > 10) {
        printf("Invalid matrix size.\n");
        return 0;
    }

    printf("Enter the matrix elements:\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            scanf("%d", &a[i][j]);
        }
    }

    do {
```

```

printf("\n1. Sum of all elements\n");
printf("2. Sum of each row\n");
printf("3. Transpose of matrix\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);

if (choice == 1) {
    int sum = 0;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            sum += a[i][j];
    printf("Sum of all elements = %d\n", sum);
}
else if (choice == 2) {
    printf("Row sums:\n");
    for (int i = 0; i < rows; i++) {
        rowSum[i] = 0;
        for (int j = 0; j < cols; j++)
            rowSum[i] += a[i][j];
        printf("Sum of row %d = %d\n", i + 1, rowSum[i]);
    }
    printf("The row-sum array contains: [");
    for (int i = 0; i < rows; i++) {
        printf("%d", rowSum[i]);
        if (i < rows - 1) printf(", ");
    }
    printf("]\n");
}
else if (choice == 3) {
    printf("Transpose of the matrix:\n");
    for (int j = 0; j < cols; j++) {
        for (int i = 0; i < rows; i++)
            printf("%d\t", a[i][j]);
        printf("\n");
    }
}
else if (choice == 4) {
    printf("Exiting the program.\n");
}
else {
    printf("Invalid choice. Please enter 1 to 4.\n");
}
} while (choice != 4);

return 0;
}

```

Prepared Test Cases

Test	Matrix	Operations	Expected Result	Purpose
1	2x3: 1 2 3 / 4 5 6	1, 2, 3, 4	Total=21; rows=[6,15]; transpose=1 4 / 2 5 / 3 6	Given sample / normal case
2	1x1: 7	1, 2, 3, 4	Total=7;	Smallest practical

			rows=[7]; transpose=7	matrix
3	3x2: 2 4 / 6 8 / 10 12	1, 2, 3, 4	Total=42; rows=[6,14,22]; transpose=2 6 10 / 4 8 12	Rectangular matrix
4	2x2: -1 5 / 3 -2	1, 2, 3, 4	Total=5; rows=[4,1]; transpose=-1 3 / 5 -2	Negative values
5	2x2: 1 2 / 3 4	9, 4	Invalid choice message, then exit	Invalid menu choice

Compilation and Execution

The program was compiled using GCC and executed against the prepared test cases. The outputs below are shown in a terminal-style format so the testing process is easy to verify.

Test Case 1 - Sample matrix (2 x 3)

```
Terminal - HD0C Day 31

$ gcc program.c -o hdoc31
$ ./hdoc31 # input: 2 3 1 2 3 4 5 6 1 2 3 4
Enter the number of rows and columns: Enter the matrix elements:

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
Enter your choice: Sum of all elements = 21

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
Enter your choice: Row sums:
Sum of row 1 = 6
Sum of row 2 = 15
The row-sum array contains: [6, 15]

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
Enter your choice: Transpose of the matrix:
1      4
2      5
3      6

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
Enter your choice: Exiting the program.
```

Status: PASS - actual output matches the expected behaviour.

Test Case 2 - Single-element matrix

```
Terminal - HD0C Day 31

$ gcc program.c -o hdoc31
$ ./hdoc31 # input: 1 1 7 1 2 3 4
Enter the number of rows and columns: Enter the matrix elements:

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
Enter your choice: Sum of all elements = 7

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
Enter your choice: Row sums:
Sum of row 1 = 7
The row-sum array contains: [7]

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
Enter your choice: Transpose of the matrix:
7

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
Enter your choice: Exiting the program.
```

Status: PASS - actual output matches the expected behaviour.

Test Case 3 - Rectangular matrix (3 x 2)

```
Terminal - HD0C Day 31

$ gcc program.c -o hdoc31
$ ./hdoc31 # input: 3 2 2 4 6 8 10 12 1 2 3 4
Enter the number of rows and columns: Enter the matrix elements:

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
Enter your choice: Sum of all elements = 42

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
Enter your choice: Row sums:
Sum of row 1 = 6
Sum of row 2 = 14
Sum of row 3 = 22
The row-sum array contains: [6, 14, 22]

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
Enter your choice: Transpose of the matrix:
2      6      10
4      8      12

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
Enter your choice: Exiting the program.
```

Status: PASS - actual output matches the expected behaviour.

Test Case 4 - Matrix with negative values

```
Terminal - HD0C Day 31

$ gcc program.c -o hdoc31
$ ./hdoc31 # input: 2 2 -1 5 3 -2 1 2 3 4
Enter the number of rows and columns: Enter the matrix elements:

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
Enter your choice: Sum of all elements = 5

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
Enter your choice: Row sums:
Sum of row 1 = 4
Sum of row 2 = 1
The row-sum array contains: [4, 1]

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
Enter your choice: Transpose of the matrix:
-1      3
5       -2

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
Enter your choice: Exiting the program.
```

Status: PASS - actual output matches the expected behaviour.

Test Case 5 - Invalid menu choice

```
Terminal - HD0C Day 31

$ gcc program.c -o hdoc31
$ ./hdoc31 # input: 2 2 1 2 3 4 9 4
Enter the number of rows and columns: Enter the matrix elements:

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
Enter your choice: Invalid choice. Please enter 1 to 4.

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
Enter your choice: Exiting the program.
```

Status: PASS - actual output matches the expected behaviour.

Sample Output Verification

For the matrix [1 2 3; 4 5 6], the total is 21. The row sums are 6 and 15, so the row-sum array is [6, 15]. The transpose has three rows and two columns: [1 4], [2 5], and [3 6]. This matches the sample output given in the activity.

Result

The menu-driven C program was written, compiled and tested successfully. It correctly finds the sum of all matrix elements, calculates and stores the sum of each row, displays the transpose, handles an invalid menu choice, and exits when requested.

What I Understood

This activity helped me understand that most matrix operations are based on nested loops. The outer loop normally handles rows and the inner loop handles columns. I also understood that a transpose can be displayed simply by reversing the order in which row and column indexes are accessed, while row sums need a fresh sum for every row.